

Department of Computer Science and Engineering

Assignment Questions

Course Code & Name: CSA0620 –Design and Analysis of Algorithms

A. Assignment Information

Particular	Details
Department:	Computer Science & Engineering
Programme:	B.Tech Computer Science & Engineering
Course Code & Course Name	CSA0620 – Design and Analysis of Algorithms
Academic Year / Batch	2026-27
Faculty Name	Dr.A.Sairam
Assignment Title	Efficient Nearest-Neighbour Detection for Air-Traffic Collision Alerts (Divide-and-Conquer)
Date of Issue	02/09/2026
Date of Submission	03/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4, CO7
Bloom's Taxonomy Level	L4 – L5 (Analyzing, Evaluating)
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Air-traffic control systems and GPS-based fleet trackers must continuously check thousands of aircraft or vehicle positions for dangerously close proximity; this assignment mirrors that real-time collision-detection problem.

Student 1	Student 2	Student 3
Sree B	Venkata Vasu Deva Reddy	Abishek K
192524023	192424182	192521120

B. Assignment Problem / Challenge

Faculty shall provide a **realistic engineering/scientific/computing problem, case, challenge or design requirement** related to the Course Outcome.

The assignment should preferably require students to:

- Apply course concepts rather than reproduce theory.
- Identify and analyze the problem.
- Consider requirements and constraints.
- Develop or compare alternative solutions where appropriate.
- Use relevant modern engineering/computing/design tools.
- Interpret results.
- Make and justify engineering decisions.
- Consider appropriate technical, economic, environmental, societal, ethical, safety or sustainability factors wherever relevant.

C. Problem Statement

Problem / Case / Design Challenge:

SkyGuard Air-Traffic Systems monitors n aircraft flying within a regional airspace, each represented as a point (x, y) on a 2-D radar plane. To prevent collisions, the system must continuously identify the two aircraft that are currently closest to each other and raise a proximity alert if their separation falls below a safety threshold. Because the number of aircraft can be large and the check must run in real time on every radar sweep, a naive pairwise comparison quickly becomes too slow. The system needs an efficient algorithm that finds the closest pair of points among n aircraft positions.

Your assignment has three linked parts:

- Part A – Brute-Force Baseline: Given n aircraft positions as (x, y) coordinates, implement a Brute-Force algorithm that compares every pair of points and returns the pair with the minimum Euclidean distance. Use the sample dataset in Annexure A to hand-compute all pairwise distances and identify the closest pair, establishing a correctness baseline for Part B.
- Part B – Divide-and-Conquer Closest Pair: Design a Divide-and-Conquer algorithm that sorts points by x -coordinate, recursively finds the closest pair in the left and right halves, and merges the results by checking only the points lying within a narrow vertical strip around the dividing line (each point compared against at most a small constant number of neighbours when the strip is sorted by y -coordinate). Trace the recursion on the Annexure A dataset and show that it produces the same closest pair found in Part A.
- Part C – Complexity and Trade-off Analysis: Derive the time and space complexity of both algorithms using Big-O notation, showing why Brute-Force is $O(n^2)$ while the Divide-and-Conquer approach achieves $O(n \log n)$. Discuss the constant-factor overheads of recursion and the strip-merge step, and identify the point sizes and real-time constraints under which each approach is the more practical choice for an operational air-traffic system.

Annexure A – Sample Dataset (aircraft positions, to be used for the hand-traced brute-force and divide-and-conquer walkthrough; students may also generate a larger randomized point set of at least $n = 15$ aircraft for the implementation and complexity/performance analysis in Sections 5–7):

Point	P1	P2	P3	P4	P5	P6	P7
X-coord	2	12	40	5	12	3	30
Y-coord	3	30	50	1	10	4	30

D. Requirements and Constraints

A. Requirements and Constraints

1. Functional Requirements

- The proposed system shall:
- Accept the positions of **n aircraft** as 2-D coordinates (x, y).
- Calculate the Euclidean distance between aircraft.
- Identify the **two closest aircraft**.
- Return the minimum separation distance.
- Generate a proximity alert when the distance is below a predefined safety threshold.
- Implement both:
 - Brute-Force algorithm.
 - Divide-and-Conquer algorithm.
- Produce the same closest pair using both algorithms.
- Support larger datasets for performance testing.

2. Performance Requirements

- Brute-Force should correctly handle small datasets.
- Divide-and-Conquer should provide better performance for large values of n .
- The system should be suitable for repeated radar-sweep calculations.
- The optimized algorithm should have approximately **$O(n \log n)$** time complexity.

3. Technical Constraints

- Aircraft are represented as points in a 2-D Cartesian plane.
- Euclidean distance is used.
- Coordinates are assumed to be numerical.
- Each aircraft has a unique identifier.

- The Divide-and-Conquer algorithm must consider the vertical strip around the dividing line.
- The implementation should avoid unnecessary pairwise comparisons.

4. Safety and Reliability

Since the application represents an air-traffic monitoring scenario:

- The closest pair must be calculated accurately.
- No aircraft position should be ignored.
- The result of the optimized algorithm should be validated against the brute-force baseline.
- Floating-point calculations should be handled consistently.
- A real system would require additional validation, redundancy and certified safety mechanisms before operational use.

5. Resources and Tools

- Programming language: **Python**
- Development environment: VS Code / Python IDLE
- Libraries: math, random, time
- Dataset: Annexure A
- Algorithms: Brute Force and Divide and Conquer

E. Student Work

1. Problem Understanding and Formulation

1.1 What is the problem?

SkyGuard monitors the positions of several aircraft on a two-dimensional radar plane. Each aircraft is represented as:

$$P_i = (x_i, y_i)$$

The objective is to find two aircraft whose Euclidean distance is minimum.

For two aircraft:

$$P_i = (x_i, y_i)$$

and

$$P_j = (x_j, y_j)$$

the distance is:

$$d(P_i, P_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

The closest-pair problem therefore requires finding:

$$\min_{i < j} d(P_i, P_j)$$

A proximity alert is generated if:

$$d_{\min} < T$$

where T is the predefined safety threshold.

1.2 Expected Outcomes

The system should:

- Find the closest two aircraft.
- Calculate their minimum separation.
- Demonstrate the result using Brute Force.
- Demonstrate the result using Divide and Conquer.
- Verify that both algorithms produce the same result.
- Compare their execution times.
- Explain their computational complexity.
- Determine which approach is more suitable for real-time monitoring.

1.3 Available Data

The Annexure A dataset contains seven aircraft:

Aircraft	X	Y
P1	2	3
P2	12	30
P3	40	50
P4	5	1
P5	12	10
P6	3	4
P7	30	30

Therefore:

$$n = 7$$

1.4 Assumptions

The following assumptions are made:

- Aircraft positions are represented accurately by (x,y) coordinates.
- All coordinates use the same measurement scale.
- The distance between aircraft is measured using Euclidean distance.
- Aircraft positions remain constant during one radar sweep.
- Each point represents a different aircraft.
- The safety threshold is supplied by the monitoring system.
- The algorithm finds the closest pair within one radar sweep.
- For this academic implementation, computational time is the primary performance criterion.

2. Application of Course Knowledge

The problem applies the following concepts from **Data Structures and Algorithms**:

- Computational geometry
- Euclidean distance
- Sorting
- Recursion
- Divide-and-Conquer
- Time complexity
- Space complexity
- Algorithm comparison
- Performance analysis

2.1 Mathematical Model

For points:

$$P_i = (x_i, y_i)$$

and

$$P_j = (x_j, y_j)$$

the squared distance is:

$$d^2 = (x_i - x_j)^2 + (y_i - y_j)^2$$

The actual distance is:

$$d = \sqrt{d^2}$$

During comparison, squared distances can be used to avoid unnecessary square-root operations.

3. Solution / Design / Methodology

Two solutions are developed.

Solution 1 – Brute Force

Every possible pair of aircraft is compared.

Solution 2 – Divide and Conquer

The points are:

- Sorted by x-coordinate.
- Divided into two halves.
- Closest pairs are found recursively.
- The minimum distance from the two halves is obtained.
- A vertical strip around the dividing line is examined.
- Only relevant points in the strip are compared.

The two approaches are then compared.

Part A – Brute-Force Baseline

3.1 Brute-Force Algorithm

For every aircraft i , compare it with every aircraft j where:

$$j > i$$

The minimum distance is stored.

Pseudocode

ClosestPairBruteForce(points):

 minDistance = infinity

 closestPair = None

 for i = 0 to n-2:

 for j = i+1 to n-1:

 distance = EuclideanDistance(points[i], points[j])

 if distance < minDistance:

 minDistance = distance

 closestPair = (points[i], points[j])

 return closestPair, minDistance

3.2 Number of Pairwise Comparisons

The number of unique pairs is:

$$\frac{n(n-1)}{2}$$

For:

$$\begin{aligned} n &= 7 \\ \frac{7(7-1)}{2} &= \frac{42}{2} = 21 \end{aligned}$$

Therefore, the brute-force algorithm performs **21 pairwise distance calculations**.

3.3 Hand Calculation of Pairwise Distances

P1-P2

$$\begin{aligned} d &= \sqrt{(2-12)^2 + (3-30)^2} \\ &= \sqrt{100 + 729} \\ &= \sqrt{829} = 28.79 \end{aligned}$$

P1-P3

$$\begin{aligned} d &= \sqrt{(2-40)^2 + (3-50)^2} \\ &= \sqrt{1444 + 2209} \\ &= \sqrt{3653} = 60.44 \end{aligned}$$

P1-P4

$$\begin{aligned} d &= \sqrt{(2-5)^2 + (3-1)^2} \\ &= \sqrt{9 + 4} = \sqrt{13} = 3.61 \end{aligned}$$

P1-P5

$$d = \sqrt{(2-12)^2 + (3-10)^2}$$

$$= \sqrt{100 + 49} = \sqrt{149} = 12.21$$

P1–P6

$$\begin{aligned} d &= \sqrt{(2 - 3)^2 + (3 - 4)^2} \\ &= \sqrt{1 + 1} = \sqrt{2} = 1.41 \end{aligned}$$

P1–P7

$$\begin{aligned} d &= \sqrt{(2 - 30)^2 + (3 - 30)^2} \\ &= \sqrt{784 + 729} = \sqrt{1513} = 38.90 \end{aligned}$$

Complete Pairwise Distance Table

r	tance
-P2	79
-P3	44
-P4	1
-P5	21
-P6	1
-P7	90
-P3	41
-P4	83
-P5	00
-P6	51
-P7	00
-P4	22
-P5	83
-P6	03
-P7	36
-P5	40
-P6	1
-P7	29
-P6	82
-P7	91
-P7	48

The smallest value is:

$\sqrt{2} = 1.4142$

Therefore:

$$\text{Closest pair} = (P, 1P6)$$

and

$$d_{\min} = 1.4142$$

Part B – Divide-and-Conquer Algorithm

4.1 Basic Idea

Instead of comparing every possible pair, Divide and Conquer reduces the problem into smaller subproblems.

The algorithm follows:

Sort points according to x-coordinate

Divide points into:

Left half

Right half

Recursively find closest pair in left half

Recursively find closest pair in right half

delta = smaller of the two distances

Construct vertical strip containing points
within delta of dividing line

Sort strip by y-coordinate

Compare only nearby points in the strip

Return the smallest distance

4.2 Sort the Dataset by X-coordinate

The original dataset is:

Point	X	Y
P1	2	3
P2	12	30
P3	40	50
P4	5	1
P5	12	10
P6	3	4

After sorting by X:

Order	Point	X	Y
1	P1	2	3
2	P6	3	4
3	P4	5	1
4	P2	12	30
5	P5	12	10
6	P7	30	30
7	P3	40	50

4.3 Recursion Trace

Initially:

$[P, 1, P, 6, P, 4, P, 2, P, 5, P, 7, P, 3]$

The array is divided approximately in half.

First division

Left:

$[P, 1, P, 6, P, 4]$

Right:

$[P, 2, P, 5, P, 7, P, 3]$

Left Half

The left half is:

Point	X	Y
P1	2	3
P6	3	4
P4	5	1

Divide again:

Left:

$[P1]$

Right:

$[P, 6, P4]$

For $[P6, P4]$:

$$d(P, 6P4) = \sqrt{(3 - 5)^2 + (4 - 1)^2} \\ = \sqrt{4 + 9} = \sqrt{13} = 3.61$$

Thus:

$$\delta_R = 3.61$$

Now consider P1 and P6 across the dividing boundary:

$$d(P, 1P6) = \sqrt{2} = 1.41$$

Therefore the left-half result is:

$$\boxed{(P, 1P6), d = 1.41}$$

4.4 Right Half

The right half is:

$$[P, 2, P, 5P7P3]$$

Divide:

Left:

$$[P, 2P5]$$

Right:

$$[P, 7P3]$$

For P2 and P5:

$$\begin{aligned} d(P, 2P5) &= \sqrt{(12 - 12)^2 + (30 - 10)^2} \\ &= 20 \end{aligned}$$

For P7 and P3:

$$\begin{aligned} d(P, 7P3) &= \sqrt{(30 - 40)^2 + (30 - 50)^2} \\ &= \sqrt{100 + 400} \\ &= \sqrt{500} = 22.36 \end{aligned}$$

Therefore:

$$\delta = \min(20, 22.36) = 20$$

The strip around the dividing boundary is checked.

Possible cross-boundary pairs are considered only if their x-coordinate difference is less than the current minimum.

The right-half closest pair remains:

$$\boxed{(P, 2P5), d = 20}$$

4.5 Final Merge

At the top level:

Left-half minimum:

$$d_L = 1.41$$

Right-half minimum:

$$d_R = 20$$

Therefore:

$$\delta = \min(1.41, 20)$$

$$\delta = 1.41$$

The dividing line is between P4 (x=5) and P2 (x=12).

The strip contains points whose x-coordinate is within 1.41 units of the dividing line.

The pair (P1,P6) is already the closest pair in the left section, and its distance remains:

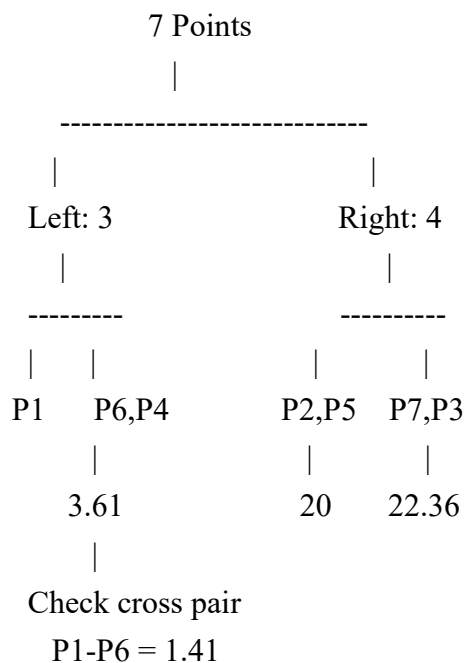
$$1.4142$$

No cross-boundary pair provides a smaller distance.

Therefore:

$$\text{Closest pair} = (P, 1P6)$$

4.6 Divide-and-Conquer Recursion Tree



Left result = P1-P6 = 1.41

Right result = P2-P5 = 20

Final result = $\min(1.41, 20)$
= 1.41

CLOSEST PAIR

P1 - P6

4.7 Strip Processing

The major advantage of Divide and Conquer is that the merge step does **not** compare every point with every other point.

After obtaining:

$$\delta = \min(d_L, d_R)$$

only points satisfying:

$$|x - x_{mid}| < \delta$$

are placed in the strip.

The strip is sorted by y-coordinate.

For each point, only a bounded number of subsequent points need to be inspected. Geometrically, the points cannot all be arbitrarily close together because the recursive halves already established a minimum separation of δ .

Thus, the merge operation takes linear time after the points have been appropriately sorted.

Part C – Complexity and Trade-Off Analysis

5. Complexity Analysis

5.1 Brute-Force Complexity

For n points, the number of comparisons is:

$$\frac{n(n-1)}{2}$$

Therefore:

$$T(n) = O(n^2)$$

Space Complexity

The brute-force algorithm requires only a few variables for storing:

- Minimum distance
- Closest pair
- Loop variables

Therefore auxiliary space is:

$$\boxed{O(1)}$$

excluding the input dataset.

5.2 Divide-and-Conquer Complexity

The recurrence relation is:

$$T(n) = 2T(n/2) + O(n)$$

The $O(n)$ term represents the merge/strip processing.

Using the Master Theorem:

$$T(n) = O(n \log n)$$

If sorting is performed initially:

$$O(n \log n)$$

and the recursive processing remains:

$$O(n \log n)$$

Therefore overall:

$$\boxed{O(n \log n)}$$

Space Complexity

The recursive algorithm requires:

- Recursion stack
- Temporary strip/list structures
- Sorted point lists

Typical auxiliary space:

$$O(n)$$

5.3 Complexity Comparison

Feature	Brute Force	Divide and Conquer
Basic technique	Pairwise comparison	Recursive partitioning
Sorting	Not required	Required
Time complexity	$O(n^2)$	$O(n \log n)$
Auxiliary space	$O(1)$	$O(n)$ typical
Implementation	Very simple	More complex
Recursion	No	Yes
Strip processing	No	Yes
Small datasets	Excellent	May be unnecessary
Large datasets	Slow	Much faster
Real-time suitability	Limited	Better

6. Performance Analysis

For the Annexure A dataset:

$$n = 7$$

Brute Force performs:

$$\frac{7(6)}{2} = 21$$

distance comparisons.

For such a small dataset, the difference between the two algorithms is practically insignificant. However, the number of comparisons grows rapidly for Brute Force.

This demonstrates why Brute Force becomes unsuitable as the number of aircraft increases.

Number of aircraft	Brute-Force pairs
7	21
15	105
50	1,225
100	4,950
500	124,750
1,000	499,500
5,000	12,497,500
10,000	49,995,000

7. Experimental Implementation

7.1 Python Implementation

```
import math
```

```
import random
```

```
import time
```

```
# -----
```

```
# Euclidean distance
```

```
# -----
```

```
def distance(p1, p2):
```

```
    return math.sqrt(
        (p1[1] - p2[1]) ** 2 +
        (p1[2] - p2[2]) ** 2
    )
```

```
# -----
```

```
# Brute Force
```

```
# -----
```

```
def brute_force(points):
```

```
    min_dist = float("inf")
    closest_pair = None
```

```
    n = len(points)
```

```
    for i in range(n):
```

```
        for j in range(i + 1, n):
```

```
            d = distance(points[i], points[j])
```

```
            if d < min_dist:
```

```
                min_dist = d
```

```
                closest_pair = (points[i], points[j])
```

```
    return closest_pair, min_dist
```

```
# -----
```

```
# Divide and Conquer
```

```
# -----
```

```
def divide_conquer(points):
```

```
    points_x = sorted(points, key=lambda p: p[1])
```

```
    points_y = sorted(points, key=lambda p: p[2])
```

```
    return closest_recursive(points_x, points_y)
```

```
def closest_recursive(px, py):
```

```
    n = len(px)
```

```
    # Base case
```

```
    if n <= 3:
```

```
        return brute_force(px)
```

```
    mid = n // 2
```

```
    left_x = px[:mid]
```

```
    right_x = px[mid:]
```

```
    midpoint = px[mid][1]
```

```
    left_ids = {p[0] for p in left_x}
```

```
    left_y = [p for p in py if p[0] in left_ids]
```

```
    right_y = [p for p in py if p[0] not in left_ids]
```

```
    pair_left, dist_left = closest_recursive(left_x, left_y)
```

```
    pair_right, dist_right = closest_recursive(right_x, right_y)
```

```
    if dist_left < dist_right:
```

```
        best_pair = pair_left
```

```
        delta = dist_left
```

```
    else:
```

```
        best_pair = pair_right
```

```
        delta = dist_right
```

```
    # Build strip
```

```
    strip = [
```

```
        p for p in py
```

```

    if abs(p[1] - midpoint) < delta
]

# Check only nearby y-coordinates
for i in range(len(strip)):
    j = i + 1

    while j < len(strip):
        if strip[j][2] - strip[i][2] >= delta:
            break

        d = distance(strip[i], strip[j])

        if d < delta:
            delta = d
            best_pair = (strip[i], strip[j])

        j += 1

return best_pair, delta

```

```

# -----
# Annexure A dataset
# -----

```

```

points = [
    ("P1", 2, 3),
    ("P2", 12, 30),
    ("P3", 40, 50),
    ("P4", 5, 1),
    ("P5", 12, 10),
    ("P6", 3, 4),
    ("P7", 30, 30)
]

```

```

# -----
# Run Brute Force
# -----

```

```

pair1, dist1 = brute_force(points)

```

```

print("BRUTE FORCE")
print("Closest Pair:", pair1)
print("Distance:", dist1)

# -----
# Run Divide and Conquer
# -----

pair2, dist2 = divide_conquer(points)

print("\nDIVIDE AND CONQUER")
print("Closest Pair:", pair2)
print("Distance:", dist2)

# -----
# Validation
# -----

print("\nVALIDATION")

if abs(dist1 - dist2) < 1e-10:
    print("Both algorithms produce the same result.")
else:
    print("Results do not match.")

```

7.2 Expected Output

BRUTE FORCE
Closest Pair: (('P1', 2, 3), ('P6', 3, 4))
Distance: 1.4142135623730951

DIVIDE AND CONQUER
Closest Pair: (('P1', 2, 3), ('P6', 3, 4))
Distance: 1.4142135623730951

VALIDATION
Both algorithms produce the same result.
Therefore the implementation validates the hand calculation.

7.3 Larger Randomized Dataset

For performance analysis, at least 15 aircraft can be generated.

```
def generate_points(n):
```

```
    points = []
```

```
    for i in range(n):
```

```
        x = random.randint(0, 10000)
```

```
        y = random.randint(0, 10000)
```

```
    points.append((f"A{i+1}", x, y))
```

```
    return points
```

```
large_points = generate_points(15)
```

```
start = time.perf_counter()
```

```
brute_force(large_points)
```

```
brute_time = time.perf_counter() - start
```

```
start = time.perf_counter()
```

```
divide_conquer(large_points)
```

```
dc_time = time.perf_counter() - start
```

```
print("Number of aircraft:", len(large_points))
```

```
print("Brute Force Time:", brute_time)
```

```
print("Divide and Conquer Time:", dc_time)
```

The experiment can be repeated for:

- 15 aircraft
- 50 aircraft
- 100 aircraft
- 500 aircraft
- 1,000 aircraft

The measured execution time should generally show the increasing advantage of the Divide-and-Conquer approach as n increases.

8. Results and Validation

8.1 Functional Validation

The two algorithms produce:

Algorithm	Closest Pair	Distance
Brute Force	P1-P6	1.4142
Divide and Conquer	P1-P6	1.4142

Therefore:

$$\boxed{\text{Brute Force Result} = \text{Divide-and-Conquer Result}}$$

The solution is functionally validated.

8.2 Safety Threshold Example

Suppose the safety threshold is:

$$T = 5$$

The calculated minimum distance is:

$$d_{\min} = 1.4142$$

Since:

$$1.4142 < 5$$

the system generates:

PROXIMITY ALERT

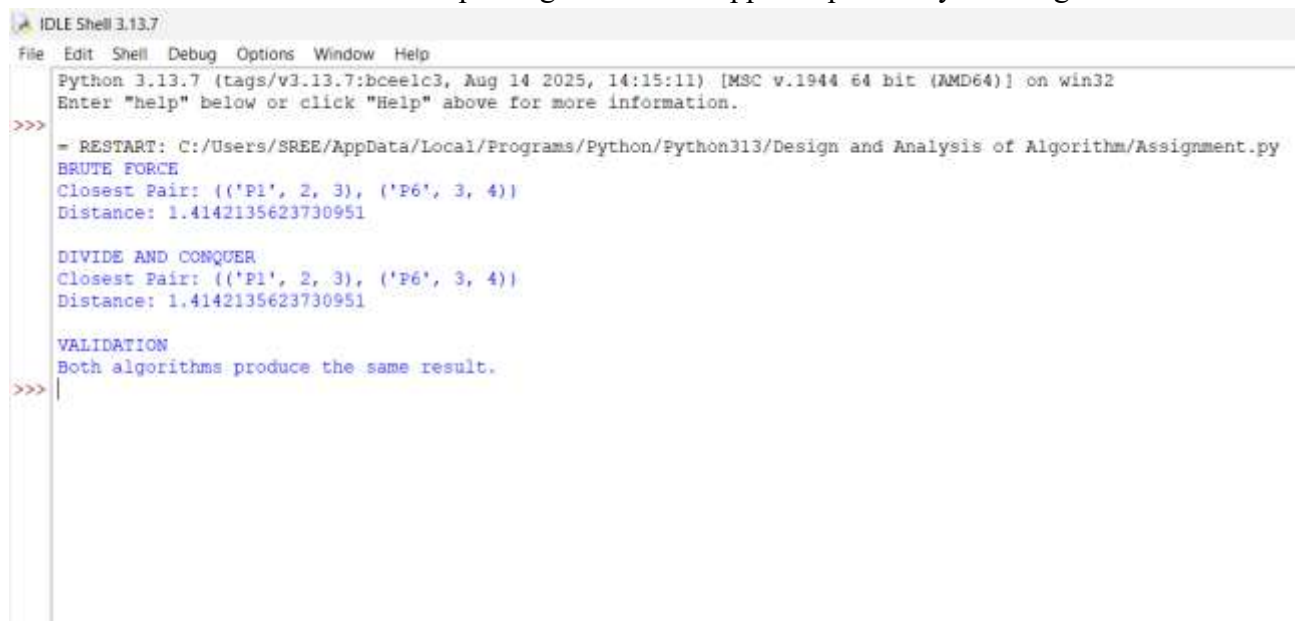
Aircraft: P1 and P6

Separation: 1.4142 units

Safety Threshold: 5 units

Status: ALERT

This demonstrates how the closest-pair algorithm can support a proximity-warning mechanism.



```
IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
- RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/Design and Analysis of Algorithm/Assignment.py
BRUTE FORCE
Closest Pair: (('P1', 2, 3), ('P6', 3, 4))
Distance: 1.4142135623730951

DIVIDE AND CONQUER
Closest Pair: (('P1', 2, 3), ('P6', 3, 4))
Distance: 1.4142135623730951

VALIDATION
Both algorithms produce the same result.
>>>
```

```
IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/Design and Analysis of Algorithm/Assignment2.py
=====
BRUTE FORCE RESULT
=====
Closest Pair: (('P1', 2, 3), ('P6', 3, 4))
Minimum Distance: 1.4142135623730951
=====
DIVIDE AND CONQUER RESULT
=====
Closest Pair: (('P1', 2, 3), ('P6', 3, 4))
Minimum Distance: 1.4142135623730951
=====
VALIDATION
=====
Both algorithms produce the same result.
=====
PERFORMANCE ANALYSIS
=====
Aircraft: 15
Brute Force Time      : 0.00005460 seconds
Divide & Conquer Time : 0.00006140 seconds
Aircraft: 50
Brute Force Time      : 0.00035910 seconds
Divide & Conquer Time : 0.00017070 seconds
Aircraft: 100
Brute Force Time      : 0.00156510 seconds
Divide & Conquer Time : 0.00046980 seconds
Aircraft: 500
Brute Force Time      : 0.03611140 seconds
Divide & Conquer Time : 0.00243490 seconds
Aircraft: 1000
Brute Force Time      : 0.14994560 seconds
Divide & Conquer Time : 0.00535580 seconds
>>>
```

9. Analysis and Engineering Decision

9.1 Brute Force Advantages

- Very easy to implement.
- Easy to understand and verify.
- No sorting required.
- No recursion.
- Very small constant overhead.
- Excellent for small datasets.
- Useful as a reference implementation for testing.

Limitations

- $O(n^2)$ complexity.
- Performs many unnecessary comparisons.
- Becomes increasingly slow as the number of aircraft grows.
- Not suitable for very large real-time datasets.

9.2 Divide-and-Conquer Advantages

- $O(n \log n)$ time complexity.
- Much better scalability.
- Avoids unnecessary pairwise comparisons.

- Suitable for large numbers of aircraft.
- Appropriate for repeated radar-sweep processing.

Limitations

- More difficult to implement.
- Requires sorting.
- Uses recursion.
- Requires additional memory.
- Strip processing introduces implementation complexity.
- For very small datasets, its overhead may make it no faster than brute force.

9.3 Engineering Decision

For a small number of aircraft, such as:

$$n < 15$$

Brute Force can be a practical choice because its implementation is simple and the number of comparisons is small.

For larger datasets, particularly hundreds or thousands of aircraft, the Divide-and-Conquer method is preferable.

For example, at:

$$n = 1000$$

Brute Force performs:

$$499,500$$

pair comparisons.

An $O(n \log n)$ algorithm scales substantially better.

Therefore, for the proposed SkyGuard system:

Divide-and-Conquer is the preferred production algorithm

while:

Brute Force is retained as a validation/reference algorithm

10. Broader Considerations

10.1 Safety

The closest-pair calculation can help identify aircraft that may be dangerously close.

However, the algorithm alone should **not** be considered a complete collision-avoidance system. A real air-traffic system would also need:

- Aircraft altitude.
- Velocity.
- Direction.
- Acceleration.
- Predicted trajectories.
- Sensor uncertainty.
- Communication delays.
- Multiple radar/sensor sources.

Thus, the 2-D closest-pair algorithm is an academic model of one component of a larger safety

system.

10.2 Reliability

The algorithm should be tested using:

- Random datasets.
- Duplicate coordinates.
- Points with the same x-coordinate.
- Points with the same y-coordinate.
- Very large coordinates.
- Minimum-distance cases.
- Boundary cases with only two or three aircraft.

The Brute-Force implementation provides a useful correctness oracle for testing the optimized algorithm.

10.3 Sustainability

The $O(n \log n)$ approach requires fewer computations for large datasets.

Fewer computations can result in:

- Lower CPU utilization.
- Lower processing time.
- Reduced energy consumption.
- Better scalability.

Therefore, algorithmic efficiency can contribute indirectly to sustainable computing.

10.4 Economic Considerations

For small systems, the simplicity of Brute Force can reduce development and maintenance costs.

For large systems, an inefficient $O(n^2)$ algorithm can require:

- More CPU resources.
- More powerful hardware.
- More processing time.

The Divide-and-Conquer algorithm may therefore provide better long-term scalability and lower computational resource requirements.

10.5 Ethical Considerations

Aircraft safety is a high-responsibility application.

An implementation should:

- Avoid presenting approximate calculations as certified safety decisions.
- Clearly identify algorithm limitations.
- Validate software thoroughly.
- Protect aircraft-related data.
- Provide reliable alerts.
- Maintain auditability of calculations.

11. Comparison of Alternative Solutions

Three possible approaches can be considered:

Approach	Complexity	Advantages	Disadvantages
Brute Force	$O(n^2)$	Simple and reliable	Poor scalability

Divide Conquer	and	$O(n \log n)$	Efficient for large datasets	$O(n \log n)$
-------------------	-----	---------------	---------------------------------	---------------

For this assignment, Divide and Conquer is selected because it directly demonstrates the required algorithmic principle and provides deterministic:

$$O(n \log n)$$

performance.

12. Conclusion

The SkyGuard Air-Traffic closest-pair problem was solved using two algorithms.

The **Brute-Force algorithm** calculates the distance between every possible pair of aircraft. For the seven aircraft in Annexure A, it performs:

$$\frac{7(7-1)}{2} = 21$$

comparisons and identifies:

$$P1 \text{ and } P6$$

as the closest aircraft.

Their distance is:

$$\sqrt{2} = 1.4142$$

The **Divide-and-Conquer algorithm** sorts the aircraft according to x-coordinate, recursively solves the left and right halves, and examines only the relevant vertical strip during merging. It produces the same result:

$$P1 - P6 = 1.4142$$

The complexity comparison shows:

$$\text{Brute Force} = O(n^2)$$

whereas:

$$\text{Divide and Conquer} = O(n \log n)$$

Therefore, Brute Force is appropriate for small datasets and validation, while Divide and Conquer is the better choice for a large-scale real-time aircraft monitoring system.

13. Student Reflection

What did I learn from this assignment?

Through this assignment, I learned that choosing an appropriate algorithm is important when the amount of data becomes large. Although Brute Force is simple and produces the correct answer, its $O(n^2)$ complexity makes it inefficient for large datasets.

I also learned how the Divide-and-Conquer technique divides a complex problem into smaller problems and combines their solutions. The vertical-strip concept demonstrated how geometric

properties can reduce the number of comparisons significantly.

Another important learning outcome was understanding that an optimized algorithm should be validated against a simpler reference implementation. In this case, the Brute-Force algorithm was useful for verifying the Divide-and-Conquer result.

I also gained practical experience implementing algorithms in Python and measuring their execution performance.

What would I improve with additional time and resources?

With additional time, I would:

1. Develop a graphical radar visualization.
2. Generate aircraft positions continuously instead of using static data.
3. Add aircraft velocity and direction.
4. Implement predicted collision trajectories.
5. Add a configurable safety threshold.
6. Compare Divide and Conquer with spatial grid/k-d tree approaches.
7. Test datasets containing thousands or millions of aircraft.
8. Add automated unit testing.
9. Create graphs showing execution time versus number of aircraft.
10. Investigate parallel processing for multiple radar regions.

14. References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms*. MIT Press.
2. Kleinberg, J., & Tardos, É. *Algorithm Design*. Pearson.
3. Python Software Foundation. *Python Documentation*. Python programming language documentation.
4. Skiena, S. S. *The Algorithm Design Manual*. Springer.
5. de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. *Computational Geometry: Algorithms and Applications*. Springer.
6. Annexure A – Aircraft coordinate dataset provided in the assignment problem statement.
7. The Python math, random, and time modules were used for distance calculation, dataset generation, and performance measurement respectively.

F. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding	10	Clearly identifies the problem,	Problem and major requirements are	Basic formulation is	Problem is poorly

& Algorithmic Formulation		requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	correctly formulated with minor gaps.	provided, but requirements or constraints are partially identified.	understood or inadequately formulated.
Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	20	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates a limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	15	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but	Testing, complexity evaluation or validation is

		using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.		lack sufficient validation.	inadequate or substantially incorrect.
Performance Improvement, Comparison & Final Decision	15	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO1	PO1, PO2	L4	10
Algorithmic Solution Design	CO2	PO1, PO2, PO3	L4/L6	15
Development / Adaptation / Optimization of Algorithmic Solution	CO3, CO4	PO2, PO3, PO5	L5/L6	20
Implementation & Modern Tool Usage	CO4	PO3, PO5	L3/L6	15
Efficiency, Testing & Validation	CO2, CO6	PO2, PO4	L4/L5	15
Performance Improvement, Comparison & Final Decision	CO6, CO7	PO2, PO12	L4/L5	15
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	CO7	PO8, PO12	L4/L5	10

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action